

Token-Accounting Integrity in LLM Metering: A Systematic Study of Client-Side Under-Payment

Hardik

Abstract—Usage-based pricing makes metering a security boundary for LLM inference: value reaches the client while accounting is still open, and the charge depends on a multi-category usage record that exists only after the request runs. We study the under-explored direction of that boundary, an honest provider and a dishonest client, by modelling a metered request as a lifecycle with one integrity property, $V(r) \leq N(r)$, and decomposing the failure space into state synchronization (B0), commitment timing (M1), and usage authority (M2). No new attack primitive is claimed; the mechanisms are individually known, and the contribution is their unification, formalization, measurement and defense analysis. Using a Dockerized FastAPI gateway over PostgreSQL and Redis, we check the lifecycle in TLA+ with TLC across ten architecture configurations and four invariants, with expectations fixed before the runs. Reservation and abort-safe finalization prove orthogonal: each alone secures one of solvency and accounting integrity, and only both secure both. Computing usage correctly is not the same as billing from it — an architecture that recounts accurately, stores the result, and still charges the client-declared number leaks 58.3 % of delivered value in the controlled setting and 69.0 % against a real serving stack, while server-authoritative billing removed the observed leakage. Substituting third-party serving code for our own generator left every architectural conclusion intact across sixty cells. Decoupling settlement in time produced over-serving under strictly sequential arrivals, showing that the B0 condition is not about concurrency: authorization must be atomically coupled to economic commitment on the path that authorizes service. We report defense overhead, model-level sufficiency conditions, and what the formal model does not cover.

Index Terms—usage metering, token accounting, billing integrity, quota enforcement, API security, economic security, streaming inference, accounting reconciliation, model checking, TLA+.

I. INTRODUCTION

THE billed unit of an LLM API is a token, and tokens are produced by an opaque, streamed, long-running computation. That makes the pipeline which authorizes, meters and commits a charge part of the security surface rather than an accounting detail. Existing security work on this surface runs in three directions, all of which assume the client is honest, impersonated, or irrelevant. Provider-side auditing asks whether a dishonest provider can inflate hidden token counts [1], [2], [3]. Economic-abuse work asks whether a third party can drain a victim’s budget [4], [5], [6], [7]. Attestation work asks whether an intermediary can misroute requests or forge provenance [9], [10]. The remaining quadrant is a legitimate, authenticated client that under-pays, and that is what we study (Table I).

Enrollment number 03517713524.

Artifact: testbed, attack harness, defense implementations, TLA+ model and all raw data are released with the paper.

We are deliberately conservative about what is new here. Each enabling mechanism already exists in isolation. Trusting a client-supplied value in a security decision is CWE-807 [11]. A deployed LLM gateway refunds pre-consumed quota when the final usage record never arrives after a disconnect [14], and a metered-token proxy already guards against an encoding-induced silent zero-debit [15]. Racing a shared counter is a classical web race [12]. Accordingly, B0 is presented as a known baseline and M1 and M2 as systematization and measurement contributions; we make no priority claim at the primitive level.

What is missing is not a primitive but a frame. These failures are usually discussed as unrelated bugs in unrelated systems. Placed on a single request lifecycle they are the same integrity property violated at three different points, and once that is written down, three things follow that individual bug reports do not give: the defenses separate into properties that do not substitute for one another, the exploitable attack set becomes a function of the pricing basis rather than of attacker ingenuity, and the conditions under which each defense is sufficient can be stated and checked.

Contributions.

- 1) An accounting model of a metered inference request (Section IV) connecting execution, value delivery, authorization, reservation, debit, reconciliation and refund, with an integrity property that admits legitimate refunds.
- 2) A taxonomy of three experimentally distinguishable dimensions (Section V) — synchronization, commitment timing, and usage authority — with the known, systematized and defensive boundaries stated explicitly.
- 3) A machine-checked formalization (Section VII): the lifecycle in TLA+, checked exhaustively with TLC over ten configurations and four invariants, producing counterexamples that match the measured failures and an exhaustive orthogonality result for the M1 defenses.
- 4) A reproducible cross-architecture measurement study (Sections VIII and X) over three execution topologies, three accounting families including an asynchronous event pipeline, and two independent inference data planes, with fail-closed integrity gates and independent recomputation.
- 5) Defense sufficiency conditions with their integrity and performance cost (Sections IX and XI), including a corrected condition for B0 that the asynchronous experiment forced us to strengthen.

II. THREAT MODEL AND SECURITY PROPERTY

The provider is honest. The adversary is a client with valid credentials whose goal is to receive metered inference

TABLE I

ADVERSARY DIRECTIONS IN LLM INFERENCE ACCOUNTING. PRIOR WORK COVERS THE FIRST THREE ROWS; THIS PAPER STUDIES THE FOURTH.

Adversary	Goal	Representative work	Studied here
Provider	Over-charge an honest client	CoIn [1], Invisible Tokens [2], Token Inflation [3]	no
Third party	Inflate a <i>victim's</i> bill	Denial-of-Wallet [4], [5], OWASP LLM10 [6], LLMjacking [7]	no
Intermediary	Misroute or forge provenance	AEX [9], gateway-path provenance [10]	no
Client	Under-pay for received inference	—	yes

and pay less than it owes. It controls six things, each exercised by our harness: request payloads, including any client-declared usage fields; permitted headers; request concurrency; the connection lifecycle, so it may abort at any moment; retries; and observation of its own responses and balance. It cannot compromise TLS, reach the database or Redis, read the gateway filesystem, compromise the inference backend, forge authenticated upstream metadata, or modify server code. This is an ordinary application-layer adversary rather than a symbolic cryptographic one, which matters for interpretation: every leak we report is attributable to the provider's own accounting design, not to a compromise.

A. Attacker knowledge

Our M2 harness builds each declared-usage manipulation by transforming the true server-side usage vector, which hands the attacker more information than a real client has. Rather than leave that convenience implicit, we grade every manipulation by the knowledge it actually needs. **K0** is client-observable only: the prompt sent, the response received, and the client's own balance. **K1** additionally requires provider-reported usage, whatever the architecture returns. **K2** requires oracle access to the exact server-side usage vector.

Under-reporting output or input, shaving a small constant, and declaring an inconsistent total are all K0, since a client can count its own prompt and the tokens it received. Inflating the cached-input category is K1, because the cached/uncached split is not visible in the response. Dropping hidden reasoning tokens is K2, because that count is never exposed. The headline usage-authority result rests on a K0 manipulation and does not depend on oracle knowledge; the two non-K0 manipulations are labelled and kept out of the main claim.

B. Integrity property

For a request r , let $V(r)$ be the value actually delivered, $D(r)$ the committed debits, $F(r)$ the refunds, and $N(r) = D(r) - F(r)$ the net committed debit. Under-payment and the integrity property are

$$\text{Leak}(r) = V(r) - N(r), \quad \text{Integrity}(r) : V(r) \leq N(r). \quad (1)$$

The inequality is over the *net* debit so that reconciliation remains legitimate: a client that consumes less than its reservation must get the difference back. A refund is legitimate only when it releases the unconsumed portion,

$$F(r) \leq \text{Res}(r) - V(r), \quad (2)$$

and the vulnerable “refund the whole reservation on abort” architecture violates exactly this side condition.

C. Two properties, not one

Accounting integrity is not the only thing a metering system owes, and merging the two obscures the defense analysis. We separate them:

- *Accounting integrity*, $V(r) \leq N(r)$: the client is never net-charged less than the value it received.
- *Solvency*: a request is served only when the system holds enough committed or reserved capacity to cover it under its own policy.

These are independent. A system can bill exactly for what it delivered while serving clients who cannot pay, and it can hold funds correctly and then hand them back. In the M1 dimension, reservation addresses solvency and abort-safe finalization addresses accounting integrity. Section VIII-B shows measurement and Section VII shows exhaustive checking that neither substitutes for the other.

III. BACKGROUND AND RELATED WORK

LLM APIs meter multi-category token vectors, stream output incrementally, and settle charges after the fact. Each of those properties has a literature, and the gap we occupy sits between them.

a) *Provider-side accounting*: CoIn shows that reasoning tokens invisible to the client can be counted and audited only with effort [1], and the companion analysis argues that opaque LLM services need audit mechanisms because the billed quantity is unverifiable to the payer [2]. Token Inflation measures how far a dishonest provider can overcharge before detection becomes likely [3]. All three take the client's side against the provider; we invert the direction while keeping their observation that the billed quantity is runtime-produced and hard to verify.

b) *Economic abuse*: Denial-of-Wallet defines forced financial exhaustion as a threat distinct from denial-of-service in serverless settings [4], and a recent review consolidates the attack and defense literature that followed [5]. OWASP lists unbounded consumption, explicitly including Denial-of-Wallet, among the top risks for LLM applications [6]. LLM-jacking documents stolen cloud credentials being resold for inference, with the victim paying [7]. Energy-latency attacks show that inference cost itself can be driven adversarially [8]. In all of these the attacker inflates somebody else's bill; here the attacker deflates its own.

c) *Intermediaries*: Attestation and provenance work targets the gateway path: AEX provides non-intrusive multi-hop attestation for LLM APIs [9], and evidence-bound gateway-path provenance addresses forged routing claims for third-party inference [10]. These assume a dishonest intermediary rather than a dishonest endpoint.

d) *Generic mechanisms*: Relying on untrusted input in a security decision is CWE-807 [11]. Concurrent quota races are well-characterized, and Kettle’s work on web race conditions shows how narrow the exploitable window can be made in practice [12]. The escrow transactional method is the classical treatment of high-contention counter updates and is the ancestor of the reserve-and-reconcile pattern we evaluate [13]. Streaming delivery and client cancellation are specified behaviour rather than exotic conditions: server-sent events define an incremental delivery channel the client may close at any time [16], and HTTP semantics leave the consequences of an aborted response to the application [17].

e) *Serving and tokenization*: The billed quantity depends on a tokenizer. Subword segmentation [18] and Sentence-Piece [19] underlie the tokenizers deployed today, and different engines disagree on the token count for identical text, which is a source of legitimate discrepancy rather than attack. Modern serving systems batch and cache aggressively; PagedAttention and vLLM established the memory management that makes prefix reuse standard [21], and prefix reuse is precisely what makes the cached-token category observable and unstable in Section X. Our real-stack validation uses `llama.cpp` [22] serving SmolLM2-135M-Instruct [23], with tiktoken [20] among the tokenizers benchmarked.

f) *Formal methods*: We specify the lifecycle in TLA+ [24] and check it with TLC [25]. Using model checking to find architectural defects that testing misses is established industrial practice [26], and our experience matches the reported pattern: the checker’s value lay in refuting a specification we believed, not in confirming one.

g) *The gap*: Within our targeted corpus we found no study evaluating client-side under-payment under an honest-provider threat model across representative metering architectures, with a common measurement and an ablation of the enforcement primitives. This is a bounded negative from a targeted search. It does not establish that no such work exists.

IV. ACCOUNTING STATE MODEL

We model a metered inference request as a state machine so that each mechanism can be localized to a specific transition, establishing that the taxonomy is *architectural* rather than a list of unrelated implementation bugs.

A. Formal model

For a request r let $U_{\text{true}}(r)$ be the usage actually produced, $U_{\text{bill}}(r)$ the usage billed, and $P(\cdot)$ the (exact-decimal) pricing function. Write $C(r) = P(U_{\text{true}}(r))$ for the true cost, $V(r)$ for the value actually delivered to the client, $\text{Res}(r)$ for any reservation, $D(r)$ for committed debits, $F(r)$ for refunds, and

$$N(r) = D(r) - F(r)$$

for the *net committed debit*. Under-payment and the integrity property are

$$\text{Leak}(r) = V(r) - N(r), \quad \text{Integrity}(r) : V(r) \leq N(r).$$

TABLE II
STRUCTURAL DISTINCTION BETWEEN THE THREE MECHANISMS.

Mechanism	Defective element	Needs concurrency?	Closed by
B0	transition atomicity	yes (≥ 2)	atomic compare-and-decrement
M1	transition graph (abort path)	no	abort-inclusive finalization
M2	data authority at ACCOUNTED	no	server-authoritative recount

Because legitimate reconciliation exists, integrity is an inequality on the *net* debit; a refund is legitimate only when it releases the unconsumed portion,

$$F(r) \leq \text{Res}(r) - V(r).$$

At trial level the harness independently verifies conservation,

$$\sum_{r \in T} N(r) = B_{\text{initial}} - B_{\text{final}},$$

and fails the build on any divergence.

B. Mechanism localization

a) *B0 (synchronization)*.: If two concurrent requests read the same authorization state B and both commit a decrement derived from it, one decrement is lost: $\sum_i N(r_i) < \sum_i V(r_i)$. This requires ≥ 2 concurrent transactions, which is why B0 leaks nothing at concurrency 1.

b) *M1 (commitment timing)*.: If $V(r) > 0$ in STREAMING but the exit path taken (ABORTED) does not reach RECONCILED with $N(r) \geq V(r)$, then $\text{Leak}(r) = V(r) > 0$. The defect is in the transition graph, not in a shared resource, hence it is concurrency-invariant.

c) *M2 (usage authority)*.: If the billing basis is client-controlled, $U_{\text{bill}} = T(U_{\text{true}})$ for an attacker transformation T , then $\text{Leak}(r) = P(U_{\text{true}}) - P(T(U_{\text{true}})) > 0$ whenever $P \circ T < P$. Authority is a per-request data-provenance property, hence also concurrency-invariant.

This predicts the empirical result: B0 leakage is *created* by concurrency, while M1/M2 per-request leakage is invariant to it.

V. ARCHITECTURAL TAXONOMY

The three dimensions are experimentally distinguishable rather than merely conceptual. B0 is a non-atomic authorize-then-decrement on shared credit state; it is a known baseline, used to validate the measurement rig and to supply a generic reference point. M1 is lifecycle-sensitive: it concerns which terminal paths reach a committed accounting event. M2 is authority- and pricing-sensitive: it concerns which representation of usage the billing function actually reads.

A. On LLM specificity

We do not claim these mechanisms are possible only in LLM systems. Streaming responses, trusting client-supplied values, and racing a counter all occur elsewhere. What is specific to LLM inference is that one request combines, at a single metering boundary: progressive value delivery; usage that is generated at runtime and therefore unknown at authorization time; multi-category token vectors; tokenizer- and provider-dependent usage semantics; pricing functions

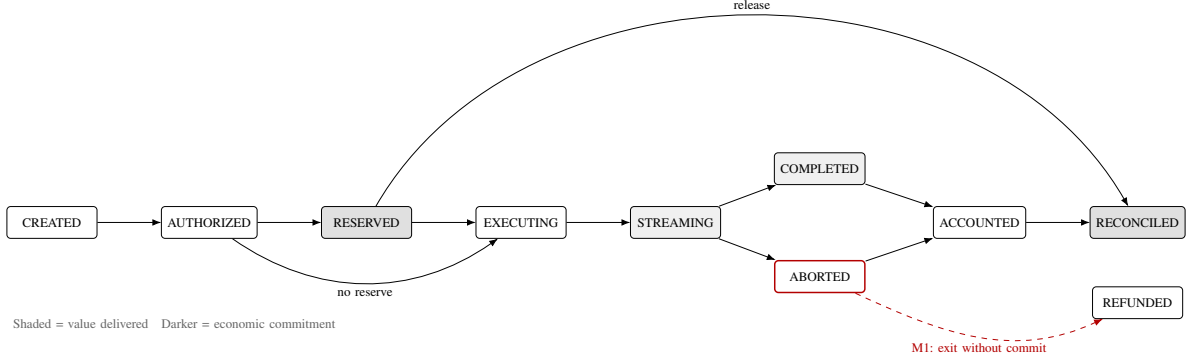


Fig. 1. Request lifecycle. Value exposure begins at the first delivered token (STREAMING); economic commitment is final only at RECONCILED. The interval between them is the exposure window that M1 exploits. B0 corrupts the AUTHORIZED→RESERVED transition (non-atomic); M2 corrupts the *data* used at ACCOUNTED.

defined over those categories; and asynchronous settlement. It is the combination we systematize, not the individual pieces.

On that basis B0 remains generic — a shared-counter race that behaves the same in any metered API — while M1 and M2 are LLM-relevant accounting dimensions. M1 is only interesting where value is delivered progressively over a long-running request, and M2 only where the billed quantity is a runtime-produced, multi-category vector priced by a category-sensitive function.

VI. SYSTEM AND EXPERIMENTAL METHODOLOGY

The testbed is a FastAPI gateway over PostgreSQL 16 in READ COMMITTED and Redis 7, running under Docker Compose on one commodity host with 16 logical CPUs and 16GB of RAM. Money is exact decimal throughout. Five M1 architectures and six M2 architectures sit behind common interfaces, so experiments compare semantics rather than unrelated code. The default generator is deterministic, which is what makes the accounting arithmetic exact; real tokenizers run on the recount path, and Section X replaces the generator entirely with third-party serving code.

Pricing is category-wise over three tiers. Leakage *efficiency* — the fraction of true value evaded — is the headline metric because it is invariant under uniform price scaling; absolute dollar figures are labelled as scenario values. The sweeps are: B0 concurrency {1, 2, 5, 10, 20, 30, 50} with 30 repetitions; M1 across architecture, abort position and concurrency {1, 5, 20, 50, 100}; M2 across architecture, manipulation and concurrency. Request volume is held constant across concurrency levels, so that volume cannot masquerade as vulnerability.

A. Integrity gates

Every summarizer recomputes each record’s leak from primitives, checks invariant-flag consistency, and verifies ledger conservation $\sum_r N(r) = B_{\text{init}} - B_{\text{final}}$ without trusting the application’s own bookkeeping. A violation fails the build. The gates are validated adversarially rather than assumed: corrupting a leak value, a debit, a served flag, a balance, or

an invariant flag, and deleting a ledger row, must each be detected, and all twelve injections were. The metamorphic suite also caught one of our own unjustified assumptions. We had asserted that leakage efficiency is invariant across price tiers; it is, but only under uniform scaling, not under a structural change to the price ratios. We corrected the property rather than suppress the failure.

B. Independent verification

The strongest M2 defense is re-implemented in a module that shares no code path with the gateway, and the two agree on all cross-validated cells (42 M2, 16 M1, 2 B0). A separate audit re-derives every reported quantity from raw data using code that imports nothing from the project, and reports zero discrepancies across 4,800 M2 records, 2,400 M1 records and 420 B0 trials. The independent checker initially reported three disagreements. They were definitional rather than defects: the checker aggregated under-payment only, while the gateway aggregates the signed billing delta, and under flat-total pricing an honest client is systematically overcharged, so the two aggregates legitimately differ. Both are now reported, and the security property uses under-payment.

C. Statistics

All statistics come from a standalone script using SciPy and statsmodels, writing a single output file that the paper cites. Many conditions here are deterministic by construction and have exactly zero variance. For those we report the constant and the complete separation rather than a *p*-value, and reserve nonparametric tests and bootstrap intervals for quantities that genuinely vary.

VII. FORMAL VERIFICATION

Measurement shows that a defect occurs on the schedules our harness produced. It cannot show that a defended architecture is safe on schedules it did not produce. We therefore encode the lifecycle of Section IV in TLA+ [24] and check it exhaustively with TLC 2.19 [25].

A. Model

One specification covers all three dimensions, with each dimension as a Boolean constant, so that the safe and unsafe architectures are the same state machine under different constants — exactly how the testbed implements them behind one interface. The constants are `AtomicDebit` for B0, `Reserves` and `AbortSafe` for M1, and `ServerAuthority` for M2. A fifth constant, `ClientMayAbort`, is a workload switch rather than an architecture property; disabling it in the B0 configurations keeps the synchronization counterexample free of commitment-timing noise, mirroring how the experiments hold the other dimensions at their safe setting.

Four invariants are checked, each the formal counterpart of a gate the summarizers already apply to every record: `ACCOUNTINGINTEGRITY` ($V(r) \leq N(r)$ for terminal r), `SOLVENCY` ($\text{balance} \geq 0$), `LEDGERCONSERVATION` (once all requests are terminal, $B_{\text{init}} - B_{\text{final}} = \sum_r N(r)$), and `REFUNDBOUNDED`, which is (2). Solvency is checked separately from accounting integrity; collapsing them would destroy the distinction the ablation exists to establish.

B. Method

Each invariant is checked in its own TLC run. TLC halts at the first violation it finds, so checking jointly would hide the fact that an M2 architecture breaks two of them. The result is a full configuration $\times$ invariant matrix (Table III): ten configurations and four invariants give 40 independent runs over 27,526 distinct states. Every expected outcome is declared in the checking script before the run, and the script exits non-zero on any disagreement. All 40 matched, with no disagreements. Runs use a single worker deliberately: with parallel workers, a run that halts at a counterexample explores a nondeterministic number of states, and the totals drifted between runs.

C. Counterexamples

TLC returns a shortest counterexample for each unsafe configuration, and each reproduces the mechanism we measured. The artifact pairs every counterexample with the measurement of the implemented architecture it abstracts; the correspondence is exact in all ten configurations.

For B0, the 21-step trace is the lost update exactly: both requests read the same balance, both write an absolute value derived from that stale read, and both then deliver and are charged. Which invariant fails is the interesting part. Accounting integrity and the refund bound both hold, because every individual record is internally consistent; what breaks is conservation of the shared balance. That is why a per-record audit cannot detect B0 while a conservation check can.

For M1, the model checks all four combinations of reservation and abort-safe finalization, not just the two the implementations ships. Reservation alone yields solvency and violates integrity. Abort-safe finalization alone yields integrity and drives the balance to -1 . Only the conjunction satisfies both. The orthogonality claim is therefore established by exhaustive

exploration rather than by the two architectures we happened to build.

For M2, the client-authoritative architecture violates accounting integrity and also the refund bound: with a reservation in place, under-billing surfaces as an over-refund. That the same defect appears in two independent invariants is a consequence of the model rather than something we set out to show, and it became visible only because the invariants are checked separately. Enabling `ServerAuthority` restores all four invariants against the same lying client; the declared value simply stops being read.

The three defenses also compose. With contention, aborts and a lying client all enabled, no invariant is violated on any reachable state, at two and at three concurrent requests.

D. Evidence labels and their limits

We use three labels and do not interchange them. `MODEL-CHECKED` means TLC explored the reachable state space exhaustively under a stated configuration. `FORMALLY ARGUED` means stated over the accounting model and reasoned about by hand. `EMPIRICALLY VERIFIED` means measured and independently recomputed. Nothing weaker than the first is ever described as proved.

Nothing here shows that a production implementation is safe. The model abstracts away the database, the network, partial failure, asynchronous reconciliation, multi-category pricing and tokenizer disagreement. It is finite, at two to three requests with unit pricing. Only safety properties are checked; no liveness property is verified. **We do not provide a mechanized refinement proof from the abstract specification to the implementation.** The correspondence between the two is an argument supported by case-by-case agreement between formal and empirical outcomes, and we do not claim more.

E. The specification was wrong first

Declaring expectations before running earned its keep immediately. Our first specification treated a reservation as a hold and set the committed debit equal to the charge at settlement, so $N = D - F$ fell below the delivered value and TLC rejected the *safe* configurations. The model was wrong, not the paper: in a reserve-and-reconcile architecture the reservation *is* the committed debit, and the refund returns its unused part. Correcting the settlement semantics made the safe configurations verify. We record the failure because a formal model that is edited until it agrees with the paper is worth nothing.

VIII. EMPIRICAL RESULTS

A. B0: synchronization

The non-atomic check-then-decrement leaks only when at least two requests overlap. Mean leakage is \$0.000 at concurrency 1, then \$0.099, \$2.871 and \$4.851 at concurrency 2, 30 and 50, growing linearly at 0.099 per additional concurrent request with $R^2 = 1.0000$ and $p = 1.9 \times 10^{-50}$. The hardened atomic compare-and-decrement leaks nothing at every level. B0 is not a contribution; it validates the rig and supplies the contrast used below. Section X revises the conclusion drawn from it.

TABLE III

TLC MODEL-CHECKING RESULTS. EACH CELL IS AN INDEPENDENT EXHAUSTIVE RUN; ‘HOLDS’ MEANS THE INVARIANT SURVIVED EVERY REACHABLE STATE AND ‘VIOLATED’ MEANS TLC RETURNED A COUNTEREXAMPLE. ALL 40 CELLS MATCHED THE EXPECTATION DECLARED BEFORE THE RUN. STATES = LARGEST REACHABLE STATE SPACE AMONG THE FOUR RUNS FOR THAT CONFIGURATION.

configuration	mech	Integrity	Solvency	Conservation	Refund bnd	states
b0 vulnerable	B0	holds	holds	VIOLATED	holds	157
b0 hardened	B0	holds	holds	holds	holds	37
m1 post completion	M1	VIOLATED	holds	holds	holds	324
m1 reserve refund on abort	M1	VIOLATED	holds	holds	VIOLATED	361
m1 no reserve settle	M1	holds	VIOLATED	holds	holds	336
m1 reserve reconcile	M1	holds	holds	holds	holds	361
m2 client	M2	VIOLATED	holds	holds	VIOLATED	361
m2 server recount	M2	holds	holds	holds	holds	361
all defenses	ALL	holds	holds	holds	holds	172
all defenses 3req	ALL	holds	holds	holds	holds	4,807

30 baseline: credit-decrement race leakage vs. concurrency (n=30 trials)

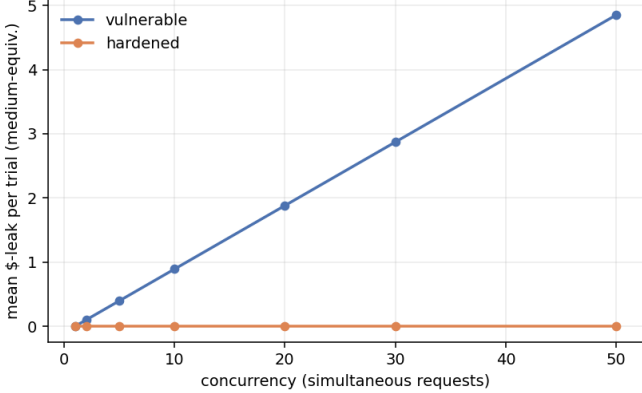


Fig. 2. B0 baseline: mean leakage per trial against concurrency, 30 repetitions per cell, deterministic generator. Leakage is created by contention here, growing linearly at 0.099 per additional concurrent request ($R^2 = 1.0000$). Section X-F shows that concurrency is not the essential ingredient: delayed settlement produces the same over-serving with strictly sequential arrivals.

B. M1: commitment timing

Leakage tracks the value delivered before the missing commit. It rises monotonically with abort position and collapses to zero at completion, when the debit finally fires: per request, \$0.0045 at roughly one token delivered, \$0.0285 at 25 %, \$0.0540 at 50 %, \$0.0960 at 90 %, and \$0.0000 at completion (Fig. 3). Request-level attack success is 1.00 at every abort position before completion. The two vulnerable architectures — debit only after completion, and refund the whole reservation on abort — are economically identical.

1) *Two orthogonal properties*: The ablation separates solvency from accounting finality. Given a budget for two requests and six issued, `no_reserve_settle` preserves accounting integrity — leakage is \$0.0000 at every abort position — yet serves all six and drives the balance to $-\$0.32$. Conversely, `reserve_refund_on_abort` holds a reservation and then refunds it away, failing both. Table IV states the resulting design rule; Section VII establishes it

M1: leakage vs. abort timing (concurrency=1, n=20 requests/cell)

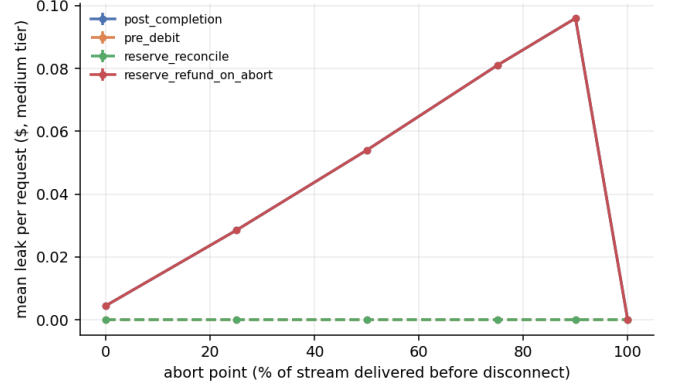


Fig. 3. M1 leakage against abort position, deterministic generator, concurrency 1, medium price tier, 20 requests per cell. The x -axis is the fraction of the response streamed before the client disconnects; the y -axis is mean leakage per request in scenario dollars, i.e. delivered value minus net committed debit. Vulnerable architectures peak just before the commit would have fired and collapse to \$0 at completion, when it does fire; safe architectures stay at \$0 throughout. All values are empirical. Error bars are within-cell standard deviations and are zero at this concurrency.

TABLE IV

M1 ORTHOGONALITY. EACH CONTROL SECURES ONE PROPERTY; ONLY THE CONJUNCTION SECURES BOTH. ALL FOUR CELLS ARE MODEL-CHECKED (TABLE III); THE DIAGONAL CELLS ARE ALSO MEASURED.

Abort-safe finalization	Reservation	
	No	Yes
No	neither property	solvency only
Yes	integrity only	both

exhaustively rather than by these two examples.

2) *Concurrency is not the mechanism*: With request volume held constant, per-request leakage was unchanged across concurrency levels 1 to 100: \$0.0528 per request at every level, identical to four decimal places, with request-level attack success exactly 1.000 for vulnerable and 0.000 for safe architectures. The honest-client control leaked nothing

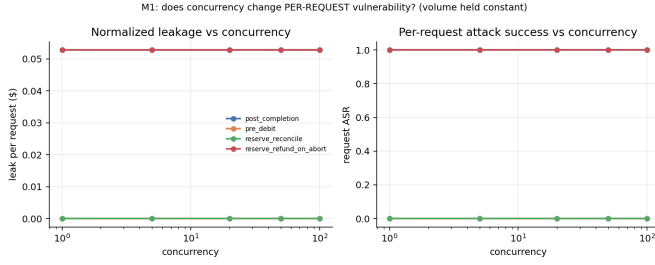


Fig. 4. M1 negative result. Request volume is held constant at 20 requests per cell while concurrency varies from 1 to 100 (x -axis, log scale); the y -axis is mean leakage per request and request-level attack success. Both are invariant across the range, which is the empirical basis for treating commitment timing as a per-request rather than a contention-driven defect. Holding volume constant is what prevents traffic level from masquerading as vulnerability.

anywhere. We do not claim invariance as a proven property of all systems under all loads; this is the measured behaviour of the tested architecture, and Section XII gives the structural reason the model predicts it.

C. M2: usage authority

The M2 result is best read as a chain. True usage is produced by the model; usage computation turns it into a vector; that vector is stored; a billing authority decides which representation to price; and a debit follows. A system can compute and store the correct usage while still billing from a different, attacker-influenced representation, and nothing in the chain flags the discrepancy.

That is not hypothetical. The `client_logged` architecture performs an accurate server-side recount, records it alongside the declared vector, and bills the declared number. Against 90% output under-reporting it leaks 58.3% of delivered value in the controlled setting, and 69.0% against the real serving stack of Section X. The manipulation is K0: a client can count the tokens it received, so no privileged knowledge is required. Every server-authoritative architecture — `server_recount`, `hybrid_reconcile` and `upstream` — drove leakage efficiency to 0.000 across all eight manipulations in the tested catalogue.

Writing U for the true usage vector, P for the pricing function and T for the attacker’s transformation of declared usage, a manipulation pays off exactly when

$$P(T(U)) < P(U), \quad (3)$$

that is, when the pricing basis reads the field T changes. Section XI develops the consequences. The exploitable set is a property of the billing function, not of attacker ingenuity.

M2’s independence from concurrency is analytic rather than discovered. The billed amount is a pure function of one request’s declared usage, computed before any balance is read, so no shared mutable state enters the billing decision and the leak of one request cannot depend on what else is in flight. The concurrency sweep confirms the derivation and serves as a check that the implementation contains no unintended shared-state dependency. It passes, and we present it as a much weaker claim than B0’s measured slope.

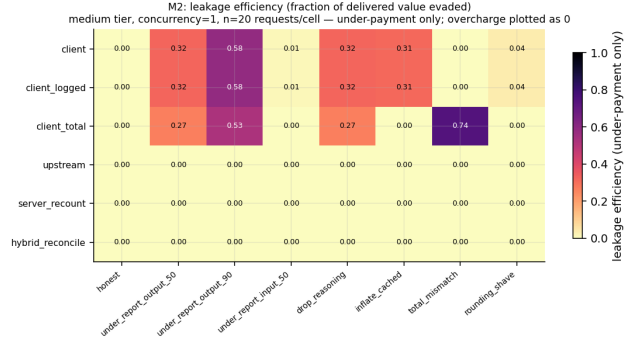


Fig. 5. M2 leakage efficiency — the fraction of delivered value evaded — by architecture (rows) against declared-usage manipulation (columns), deterministic generator, medium tier, concurrency 1, 100 requests per cell. Server-authoritative rows are uniformly zero. The exploitable set differs between category-additive and flat-total pricing, which is the empirical form of (3): a manipulation pays off only when the pricing basis reads the field it changes.

D. Lifecycle failure modes

Beyond the scripted attacks we inject nine lifecycle failures: disconnect at $t = 0$ and mid-stream, truncated stream, settle-window race, missing, delayed and duplicated usage metadata, and a recount-engine failure. Every invariant violation falls on a vulnerable architecture, the hardened architectures survive all of them, and a recount-engine failure is fail-closed — the request is rejected and nothing is served or billed.

E. A withdrawn claim

An earlier version of this work reported a detectability taxonomy and claimed that logging a server-side recount moved a leak from invisible to post-hoc reconcilable. An independent audit of our own artifact showed the claim was not supported, and we withdraw it.

Two defects were involved. The implementation assigned the detectability level from static architecture metadata rather than measuring it, so the experiment read back a label we had set. When we replaced that with a classifier reading only retained ledger evidence and blind to architecture labels, the distinction disappeared: the two architectures in question retain identical evidence, so both are equally reconcilable. Across the full dataset, 2,200 records carrying the stored “invisible” label are reconcilable or prevented on the evidence, and no record in the study qualifies as invisible at all.

The root cause is a property of the testbed rather than of the architectures: our harness records ground-truth usage for every architecture because it needs that value to compute leakage, whereas a production deployment need not retain it. Our instrumentation therefore cannot distinguish detectability classes, so we make no detectability claim. We report this as a limitation rather than build additional instrumentation to rescue the metric.

IX. DEFENSES AND SUFFICIENCY CONDITIONS

The ablation shows which primitive is load-bearing empirically. Here we state, per mechanism, conditions sufficient

TABLE V

M2 AS A TRANSFORMATION ON THE USAGE VECTOR. WHETHER A MANIPULATION PAYS OFF IS DETERMINED BY WHETHER THE PRICING BASIS READS THE FIELD T CHANGES.

architecture	pricing basis	manipulation T	vector change T(U)	leak eff.	why it succeeds / fails
client	category	honest	unchanged	+0.000	T is identity (honest)
client	category	under_report_output_50	output:60->30, total:98->68	+0.324	T lowers a category the basis prices
client	category	under_report_output_90	output:60->6, total:98->44	+0.583	T lowers a category the basis prices
client	category	under_report_input_50	input:8->4, total:98->94	+0.014	T lowers a category the basis prices
client	category	drop_reasoning	reasoning:30->0, total:98->68	+0.324	T lowers a category the basis prices
client	category	inflate_cached	output:60->30, cached_input:0->30	+0.313	T lowers a category the basis prices
client	category	total_mismatch	total:98->24	+0.000	T changes only the declared total; category basis never reads it
client	category	rounding_shave	input:8->7, output:60->57, total:98->94	+0.036	T lowers a category the basis prices
client_total	flat-total	honest	unchanged	-0.058	T is identity (honest)
client_total	flat-total	under_report_output_50	output:60->30, total:98->68	+0.266	T lowers the declared total the basis prices
client_total	flat-total	under_report_output_90	output:60->6, total:98->44	+0.525	T lowers the declared total the basis prices
client_total	flat-total	under_report_input_50	input:8->4, total:98->94	-0.014	T's change is priced at (near) the same rate
client_total	flat-total	drop_reasoning	reasoning:30->0, total:98->68	+0.266	T lowers the declared total the basis prices
client_total	flat-total	inflate_cached	output:60->30, cached_input:0->30	-0.058	T changes only subtotals; flat-total basis never reads them
client_total	flat-total	total_mismatch	total:98->24	+0.741	T lowers the declared total the basis prices
client_total	flat-total	rounding_shave	input:8->7, output:60->57, total:98->94	-0.014	T's change is priced at (near) the same rate

within the accounting model to guarantee (1). These are model-level arguments, not universal theorems about deployed systems, and we keep what the model forces separate from what our implementation happens to supply.

a) *M1, commitment timing*: Two conditions suffice. (M1-

a) *Terminal-path totality*: every terminal path — client disconnect, server exception, stream truncation — reaches a state where an accounting event for r is committed and durable. (M1-b) *Refund boundedness*: every refund satisfies (2). If a reservation was taken, (M1-b) gives $N(r) = \text{Res}(r) - F(r) \geq V(r)$; if not, (M1-a) forces the terminal event to commit $D(r) = V(r)$. Solvency is a separate condition, (M1-c): reserve atomically before execution and refuse the request when the reservation fails. The ablation cell `no_reserve_settle` satisfies (M1-a) and (M1-b) but not (M1-c), and measured exactly that — integrity held while the balance went negative.

b) *M2, usage authority*: Two conditions suffice. (M2-a) *Authority*: the usage value used for billing derives from state the attacker cannot control, $U_{\text{bill}} = U_{\text{true}}$. (M2-b) *Pricing monotonicity*: P is non-decreasing in each category. Computing U_{true} is not sufficient. An architecture that computes it correctly, records it, and bills $P(U_{\text{decl}})$ satisfies “the server recounts” while violating (M2-a), and leaks $P(U_{\text{true}}) - P(T(U_{\text{true}}))$.

c) *B0, synchronization*: The condition we originally stated was that the check balance $\geq$ cost and the decrement occur as one operation, atomic with respect to competing transactions. The asynchronous experiment in Section X showed that this is not enough, because it says nothing about *where* in the pipeline the atomic operation sits. The corrected condition is (B0-a): the authorization decision must be atomically coupled to the economic commitment *on the path that authorizes service*. Concurrency is one way to break that coupling; delayed settlement is another, and the second requires no concurrency at all.

d) *Implementation-specific caveats*: The model says “committed and durable”; our implementation achieves it with a cancellation-shielded settlement task, and a framework that cancelled that task would satisfy the architecture on paper while violating (M1-a). The upstream architecture satisfies

(M2-a) only because the provider is honest by assumption. And (B0-a) is a property rather than a mechanism: our backends obtain it by a conditional UPDATE and by an anchor-row lock respectively, which is what Section X tests.

A. Practical guidance

Table VI condenses the analysis into what an operator can act on, keeping the model-level condition separate from the implementation pattern that satisfies it. The patterns are the ones we implemented and measured; they are not claims of production safety under conditions we did not test.

B. What the strongest defense costs

We measure the M2 recount at three levels because they answer different questions.

First, the standalone tokenizer (Fig. 6). Cost is linear in input length — log-log slope 0.999 to 1.14 with $R^2 \geq 0.998$ — so it budgets as tokens times a per-token constant, roughly 0.7 $\mu\text{s}/\text{token}$ for tiktoken and 2.5 $\mu\text{s}/\text{token}$ for SentencePiece. At 16,384 tokens the engines span 10.1 to 41.8 ms, a 2.9 to 3.6 $\times$ implementation effect, and at fixed token count the workload alone changes cost by up to 3.4 $\times$.

Second, the gateway with the deterministic generator. Recount is nearly free at 1,024 tokens or fewer, retaining at least 0.96 $\times$ throughput, but costs heavily at 16,384 tokens, at 0.05 to 0.76 $\times$. Tokenization time explains this, with Spearman $\rho = -0.868$ and $p = 1.3 \times 10^{-5}$.

Third, the local real-model experiment. With real autoregressive generation the recount is well under 0.1% of end-to-end time across 128-, 1024- and 4096-token contexts: 0.014 to 0.019% at 64 new tokens, and 0.017 to 0.028% on an independent re-measurement with separately written timing code at 32 new tokens. The ratio rises as generation shortens, which is what the cost model predicts. Measured throughput ratios scatter around 1.0 with signs that flip across workloads, inside a $\pm 10\%$ noise band that is itself some 700 $\times$ larger than the effect.

The apparent expense in the second case is an artifact of a generator that costs nothing: when generation is free, tokenization is the entire request. We report all three rather

TABLE VI

PRACTICAL DEFENSE SUMMARY. THE MODEL-LEVEL CONDITION IS WHAT THE ACCOUNTING MODEL REQUIRES; THE IMPLEMENTATION PATTERN IS WHAT WE BUILT AND MEASURED. SATISFYING THE PATTERN IN A DIFFERENT SYSTEM DOES NOT BY ITSELF ESTABLISH THE CONDITION THERE.

Mech.	Failure mode	Property at risk	Model-level condition	Implementation pattern
B0	Authorization reads credit state that does not yet reflect prior commitments	Ledger conservation, solvency	Authorization atomically coupled to economic commitment on the authorizing path	Conditional <code>UPDATE ... WHERE balance >= cost</code> evaluated on the request path, not deferred to a later worker
M1	A terminal path leaves the lifecycle without a committed debit, or refunds more than the unconsumed reservation	Accounting integrity; solvency separately	Terminal-path totality plus refund boundedness (2); reservation for solvency	Cancellation-shielded settlement on every exit path, refund capped at $\text{Res}(r) - V(r)$, atomic reserve before execution
M2	Correct usage is computed and stored, but billing reads a client-influenced representation	Accounting integrity	Billing basis derived from state outside attacker control; pricing monotone per category	Server-side recount used <i>as the billing input</i> , with the declared vector retained only for reconciliation
Async	Usage event lost, duplicated, or applied after the next authorization	Accounting integrity; conservation	Durable, idempotent accounting events with bounded reconciliation lag	Idempotency key per event, at-least-once delivery with duplicate suppression, and authorization that accounts for in-flight events

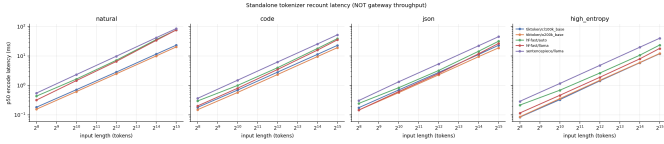


Fig. 6. Standalone tokenizer recount latency (p50, log-log axes) by engine, workload and input length, over 100 cells whose token lengths were verified exactly. Cost is linear in input length, so recount budgets as tokens times a per-token constant. This is a tokenizer microbenchmark, not a gateway throughput result: no request path, database or network is involved.

than the most convenient one, and we do not claim the recount is free. It is a real, predictable cost that is small relative to real inference in the tested configuration.

X. CROSS-ARCHITECTURE AND ASYNCHRONOUS VALIDATION

A measurement study on one deployment invites the objection that its conclusions are artifacts of that deployment. We repeated the experiments while changing only the deployment, the storage architecture, or the inference data plane; the attack harness, the architectures under test and the integrity gates are identical across all runs.

A. Execution topologies

Three topologies: a single uvicorn worker with no proxy, as control; nginx over four uvicorn workers, giving four independent event loops sharing one PostgreSQL and one Redis; and an nginx load balancer over two gateway containers. Across 96 accounting cells there were no mismatches against the control. Two genuine multi-worker defects surfaced during this work and are reported rather than hidden: the B0 runtime posture was per-process and would have reached one worker in four, and concurrent schema creation raced. Both were fixed.

B. Accounting backends

Backend A keeps a mutable balance row and obtains atomicity from a conditional `UPDATE ... WHERE balance >=`

`cost`. Backend B never mutates that row: every movement is an immutable entry in an append-only ledger and the balance is derived as the sum of entries, with the atomic reserve becoming a conditional append serialized on an anchor row.

All 17 comparison cells agreed, and we state plainly what that number contains. Only **eight** genuinely exercised both backends. Five agree for an analytic reason — the M2 billed amount is computed before any balance is read, so it cannot vary by backend — and four are B0 cells whose frozen debit module contains no reference to the backend abstraction, so they ran identical code twice. The defensible statement is that eight genuine backend-sensitive cases agreed and none disagreed. B0’s independence from the storage representation is therefore untested, and we list it as a limitation.

C. A real serving stack

Every figure so far was produced against a generator we wrote. That is what makes the accounting arithmetic exact, but it leaves an obvious objection open, so we repeated the experiments with the token source replaced by `llama.cpp`’s server [22] running `SmolLM2-135M-Instruct` [23] on CPU. This is self-hosted serving code we did not write, with a real subword tokenizer, real server-sent event streaming [16], and usage records the server produces itself. The gateway, architectures, accounting backends, settlement logic and integrity gates are unchanged, and the deterministic default remains byte-identical, with both regression suites still passing.

Across 12 M1 cells and 48 M2 cells there were no safety disagreements. Vulnerable commitment-timing architectures still leak on abort and collapse to zero at completion, reserve-and-reconcile still leaks nothing, and every server-authoritative architecture still leaks exactly 0.000 across all eight manipulations.

Attack effectiveness did move, and we report it rather than average it away. Output under-reporting at 90% yields 0.690 here against 0.583 on the deterministic generator, because `SmolLM2` emits no reasoning tokens and output therefore dominates the cost. For the same reason the K2 manipulation

TABLE VII

CROSS-TOPOLOGY VALIDATION. ACCOUNTING OUTCOMES ARE COMPARED AGAINST THE SINGLE-WORKER CONTROL. 'DIFF IN TOKENS' EXPRESSES AN M1 DIFFERENCE IN WHOLE DELIVERED TOKENS; SUB-TOKEN DIFFERENCES REFLECT DISCONNECT-DETECTION TIMING, NOT ACCOUNTING DIVERGENCE. 'DISTINCT PROCS' CONFIRMS LOAD ACTUALLY SPREAD.

mech	cell	topology	single	topology value	abs diff	diff in tokens	distinct procs	verdict
M1	post_completion @0%	multiworker	0.0045	0.0047	0.0002	0.15	4	WITHIN 1 TOKEN
M1	post_completion @25%	multiworker	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	post_completion @50%	multiworker	0.0540	0.0540	0.0000	0.01	4	WITHIN 1 TOKEN
M1	post_completion @75%	multiworker	0.0810	0.0810	0.0000	0.00	4	IDENTICAL
M1	post_completion @90%	multiworker	0.0960	0.0960	0.0000	0.00	4	IDENTICAL
M1	post_completion @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	pre_debit @0%	multiworker	-0.1020	-0.1019	0.0001	0.09	4	WITHIN 1 TOKEN
M1	pre_debit @25%	multiworker	-0.0780	-0.0780	0.0000	0.00	4	IDENTICAL
M1	pre_debit @50%	multiworker	-0.0525	-0.0525	0.0000	0.00	4	IDENTICAL
M1	pre_debit @75%	multiworker	-0.0255	-0.0255	0.0000	0.00	4	IDENTICAL
M1	pre_debit @90%	multiworker	-0.0105	-0.0105	0.0000	0.00	4	IDENTICAL
M1	pre_debit @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @0%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @25%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @50%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @75%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @90%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @0%	multiworker	0.0045	0.0045	0.0000	0.03	4	WITHIN 1 TOKEN
M1	reserve_refund_on_abort @25%	multiworker	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @50%	multiworker	0.0540	0.0540	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @75%	multiworker	0.0810	0.0810	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @90%	multiworker	0.0960	0.0960	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	post_completion @25%	distributed	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	post_completion @50%	distributed	0.0540	0.0540	0.0000	0.01	4	WITHIN 1 TOKEN
M1	post_completion @75%	distributed	0.0960	0.0960	0.0000	0.03	4	WITHIN 1 TOKEN
M1	pre_debit @25%	distributed	-0.0780	-0.0780	0.0000	0.00	4	IDENTICAL
M1	pre_debit @50%	distributed	-0.0525	-0.0525	0.0000	0.00	4	IDENTICAL
M1	pre_debit @90%	distributed	-0.0105	-0.0105	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @25%	distributed	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @50%	distributed	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @75%	distributed	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @25%	distributed	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @50%	distributed	0.0540	0.0540	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @90%	distributed	0.0960	0.0960	0.0000	0.00	4	IDENTICAL
M2	client_drop_reasoning	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client_inflate_cached	multiworker	0.313	0.313	0.0000	-	4	IDENTICAL
M2	client_rounding_shave	multiworker	0.036	0.036	0.0000	-	4	IDENTICAL
M2	client_total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_under_report_output_50	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client_under_report_output_90	multiworker	0.583	0.583	0.0000	-	4	IDENTICAL
M2	client_logged_drop_reasoning	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client_logged_inflate_cached	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_logged_rounding_shave	multiworker	0.036	0.036	0.0000	-	4	IDENTICAL
M2	client_logged_total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_logged_under_report_output_50	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client_logged_under_report_output_90	multiworker	0.583	0.583	0.0000	-	3	IDENTICAL
M2	client_total_drop_reasoning	multiworker	0.266	0.266	0.0000	-	4	IDENTICAL
M2	client_total_inflate	multiworker	-0.058	-0.058	0.0000	-	4	IDENTICAL
M2	client_total_rounding_shave	multiworker	-0.014	-0.014	0.0000	-	4	IDENTICAL
M2	client_total_total_mismatch	multiworker	0.741	0.741	0.0000	-	4	IDENTICAL
M2	client_total_under_report_output_50	multiworker	0.266	0.266	0.0000	-	4	IDENTICAL
M2	client_total_under_report_output_90	multiworker	0.525	0.525	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_drop_reasoning	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_inflate_cached	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_rounding_shave	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_under_report_output_50	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_under_report_output_90	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_drop_reasoning	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_inflate_cached	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_rounding_shave	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_under_report_output_50	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_under_report_output_90	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_drop_reasoning	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_inflate	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_rounding_shave	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_under_report_output_50	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_under_report_output_90	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_inflate	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_total_mismatch	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_under_report_output_90	distributed	0.583	0.583	0.0000	-	4	IDENTICAL
M2	client_logged_inflate	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_logged_total_mismatch	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_logged_under_report_output_90	distributed	0.583	0.583	0.0000	-	4	IDENTICAL
M2	client_total_inflate	distributed	-0.058	-0.058	0.0000	-	4	IDENTICAL
M2	client_total_total_mismatch	distributed	0.741	0.741	0.0000	-	4	IDENTICAL
M2	client_total_under_report_output_90	distributed	0.525	0.525	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_inflate	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_total_mismatch	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile_under_report_output_90	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_inflate	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_total_mismatch	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount_under_report_output_90	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_inflate	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream_total_mismatch	distributed	0.000	0.000	0.0000	-	3	IDENTICAL
M2	upstream_under_report_output_90	distributed	0.000	0.000	0.0000	-	4	IDENTICAL

that drops hidden reasoning tokens becomes completely inert: there is nothing to drop. Under flat-total pricing three cells flip sign, because on this token mix flat-total overcharges by so much that even a 90 % under-declared total still exceeds the authoritative category cost. Five of the 48 cells changed effectiveness in total; the per-cell comparison against the deterministic corpus is in the artifact. The architectural conclusion is generator-independent; the per-manipulation magnitudes are not.

D. The provider's own number is not reproducible

One property of real serving appeared that the deterministic generator cannot exhibit. The server reports a prefix-cache split, and because cached input is priced an order of magnitude below uncached input, the *honest* charge for a byte-identical request depends on server cache state. Repeating identical

TABLE VIII

CROSS-BACKEND VALIDATION. THE SAME CASES RUN AGAINST A MUTABLE BALANCE ROW AND AN APPEND-ONLY LEDGER WITH A DERIVED BALANCE, BEHIND ONE SEMANTIC INTERFACE. THE 'EVIDENCE' COLUMN STATES WHAT EACH AGREEMENT IS WORTH: GENUINE = THE CASE ACTUALLY EXERCISED BOTH BACKENDS AND COULD HAVE DISAGREED; ANALYTIC = THE M2 BILLED AMOUNT IS COMPUTED BEFORE ANY BALANCE IS READ, SO IT CANNOT VARY BY BACKEND; VACUOUS = AGREEMENT BETWEEN TWO ZERO-LEAK OUTCOMES, WHICH ANY CORRECT IMPLEMENTATION PRODUCES. B0 CASES ARE ANALYTIC-AT-BEST BECAUSE THE FROZEN B0 DEBIT MODULE DOES NOT REFERENCE THE BACKEND ABSTRACTION. ONLY THE GENUINE COUNT IS EVIDENCE OF STORAGE-ARCHITECTURE INDEPENDENCE.

case	metric	mutable	ledger	viol (mut)	viol (led)	verdict	evidence
B0/hardened/c10	leak	0.000000	0.000000	0	0	IDENTICAL	VACUOUS
B0/hardened/c50	leak	0.000000	0.000000	0	0	IDENTICAL	VACUOUS
B0/vulnerable/c10	leak	0.949500	0.949500	9	9	IDENTICAL	VACUOUS
B0/vulnerable/c50	leak	5.169500	5.169500	49	49	IDENTICAL	VACUOUS
M1/no_reserve_settle/abort50	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/no_reserve_settle/abort90	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/post_completion/abort50	leak_per_request	0.054500	0.054500	10	10	IDENTICAL	GENUINE
M1/post_completion/abort90	leak_per_request	0.095000	0.095000	10	10	IDENTICAL	GENUINE
M1/reserve_reconcile/abort50	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/reserve_reconcile/abort90	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/reserve_refund_on_abort/abort50	leak_per_request	0.054500	0.054500	10	10	IDENTICAL	GENUINE
M1/reserve_refund_on_abort/abort90	leak_per_request	0.095000	0.095000	10	10	IDENTICAL	GENUINE
M2/client_under_report_output_90	efficiency	0.5942492012779552	0.5942492012779552	10	10	IDENTICAL	ANALYTIC
M2/client_logged_under_report_output_90	efficiency	0.5942492012779552	0.5942492012779552	10	10	IDENTICAL	ANALYTIC
M2/client_total_mismatch	efficiency	0.7412140575079872	0.7412140575079872	10	10	IDENTICAL	ANALYTIC
M2/hybrid_reconcile_total_mismatch	efficiency	0.0	0.0	0	0	IDENTICAL	ANALYTIC
M2/server_recount_under_report_output_90	efficiency	0.0	0.0	0	0	IDENTICAL	ANALYTIC

TABLE IX

THE AUTHORITATIVE COST OF A BYTE-IDENTICAL REQUEST ON A REAL SERVING STACK. 'CACHED_TOKENS' IS A PREFIX-CACHE SPLIT REPORTED BY THE SERVER; BECAUSE CACHED INPUT IS PRICED 10X BELOW UNCACHED INPUT, THE PROVIDER'S OWN HONEST CHARGE FOR THE SAME REQUEST CHANGES WITH SERVER CACHE STATE. THE MOCK GENERATOR CANNOT EXHIBIT THIS.

prompt tokens	cached tokens per call	honest cost per call	spread
53	24 / 52 / 52 / 52	0.039700 / 0.027100 / 0.027100 / 0.027100	31.74%
53	35 / 52 / 52 / 52	0.034750 / 0.027100 / 0.027100 / 0.027100	22.01%
53	35 / 52 / 52 / 52	0.034750 / 0.027100 / 0.027100 / 0.027100	22.01%

requests, the reported split moved — 24 then 52 of 53 prompt tokens in one instance — and the authoritative cost moved with it by 22 to 27 %, reaching 32 % at the extreme, on all three prompts tested.

Three consequences follow, and they sharpen the threat model rather than decorate it. The provider's authoritative number is not reproducible across calls. A client cannot verify its own bill even in principle. And a client that over-declares cached tokens has genuine plausible deniability, because the true value legitimately varies. This is one stack with one prefix-cache implementation and we do not generalize it to hosted APIs, but it is the kind of accounting nondeterminism that separates LLM metering from metering a fixed-price call.

E. Asynchronous accounting

Both backends so far settle before the response completes. Real metering pipelines often do not: the gateway emits a usage event and a separate worker applies it later. We therefore added a third accounting family — an event queue with a continuously running worker and a controlled reconciliation delay D — because a claim of architectural generality that quietly assumes strong consistency is not worth much.

With an honest pipeline, asynchronous accounting is *delayed rather than lossy*. The mean exposure window between value delivery and durable debit tracks D closely: at $D = 0, 10, 50, 100, 500$ ms the measured windows are 14.0, 14.9, 54.3, 103.7 and 504.1 ms, and the leak reconciles to exactly zero. Duplicated events are suppressed idempotently across

TABLE X

B0 UNDER ASYNCHRONOUS SETTLEMENT. REQUESTS ARE ISSUED STRICTLY SEQUENTIALLY, 20 MS APART, SO NO TWO ARE EVER IN FLIGHT TOGETHER: ANY OVER-SERVING HERE CANNOT BE A CONCURRENCY RACE. OVER-SERVING INSTEAD APPEARS ONCE THE RECONCILIATION DELAY EXCEEDS THE INTER-ARRIVAL TIME, AND THE BALANCE GOES NEGATIVE. DECOUPLING SETTLEMENT CONVERTS A TIMING RACE INTO AN ARCHITECTURAL WINDOW.

delay (ms)	inter-arrival (ms)	issued	served	affordable	over-served	final balance (\$)
0	20	6	2	2	0	0.000000
10	20	6	2	2	0	0.000000
50	20	6	2	2	0	0.000000
100	20	6	3	2	1	-0.081000
500	20	6	6	2	4	-0.324000

all 25 injected duplicates. A lost event, by contrast, leaks permanently at every delay. Whether a straggler looks lost or merely late is a property of the observation horizon rather than of the architecture; we used four times the configured delay, floored at 400 ms, and the artifact records the per-cell outcome.

F. B0 revisited: the condition is not about concurrency

The asynchronous family changed one of our own conclusions, and it is the sharpest systems result in the paper.

We issued requests *strictly sequentially*, 20 ms apart, so that no two were ever in flight together. Any over-serving therefore cannot be a concurrency race. Over-serving nonetheless appears once D reaches 100 ms, and reaches four requests over a two-request budget at $D = 500$ ms, with the balance driven negative.

The mechanism is simple once seen. Authorization reads a committed balance that is correct but stale, because the debits for earlier requests are still in the queue. An atomic guarded decrement — the entire B0 defense — provides nothing if it lands after the next authorization has already read the balance. Concurrency was never the essential ingredient; it was one way of separating the check from the commitment. Delayed settlement is another, and it needs no concurrency at all.

We therefore restate the B0 condition as given in Section IX: authorization must be atomically coupled to economic commitment on the path that authorizes service. This is strictly more general than the concurrency framing we started with, and the earlier framing was too weak rather than wrong.

M2, by contrast, is untouched: leakage efficiency is identical at 0.5823 at every delay, confirming the analytic independence of usage authority from settlement timing.

XI. ECONOMIC AND PRICING ANALYSIS

The security question in M2 reduces to an economic one. With U the true usage vector, T the attacker’s transformation and P the pricing function, a manipulation is profitable exactly under (3): the pricing basis must read a field that T changes.

That has an immediate structural consequence. Under *category-additive* pricing, $P(U) = \sum_i p_i u_i$, a manipulation pays off if and only if it reduces a category the function actually reads. Declaring a total inconsistent with the subtotals is therefore inert, measured at 0.000, because category pricing never reads the declared total. Under *flat-total* pricing,

$P(U) = p \cdot \text{total}(U)$, the same manipulation becomes the strongest attack in the catalogue at 0.741, while reclassifying output into a discounted cached category becomes inert because no category is read at all. Mixed pricing over input, output, cached and reasoning categories sits between the two, and its exploitable set is determined by which categories carry distinct prices.

Two kinds of price change must be distinguished. *Uniform scaling* of every price by a constant leaves leakage efficiency unchanged, which is why we use efficiency as the headline metric and label absolute dollars as scenario values. *Structural repricing* — changing the ratios between categories, or moving between category-additive and flat-total bases — changes which transformations are profitable, and can change them qualitatively. The real-stack experiment demonstrates the second case directly: on a token mix with no reasoning tokens, three flat-total cells reverse sign, because the pricing basis overcharges by more than the manipulation evades. The full price-tier sensitivity table is in the artifact; its summary is that efficiency is identical across a uniform $10\times$ scaling and moves only when the ratio between categories changes.

We deliberately stop short of a revenue-impact model. Estimating aggregate exposure would require assuming an attacker population and a request volume, and the resulting figure would be an invention dressed as a measurement. What the analysis does support is a design rule: the pricing basis determines the attack surface, so it should be chosen with that in mind rather than for billing convenience alone.

XII. DISCUSSION

Three dimensions of one accounting boundary behave differently, and the difference follows from which element of the lifecycle is defective. Transition atomicity, B0, is a property of shared state and fails whenever the check is separated from the commitment. The transition graph, M1, and data authority, M2, are per-request properties and showed no concurrency dependence in the tested design.

The practical consequences are three. First, an operator cannot use traffic level as a proxy for exposure; two of the three dimensions do not scale with load at all, and the third fails under sequential arrivals if settlement lags. Second, integrity and solvency need different controls, and a system with only one of them will look correct in whichever dimension it happens to be measured. Third, in the evaluated architectures a correct recount did not prevent leakage unless it was authoritative for billing. The defenses themselves are ordinary; the contribution is knowing which one is load-bearing, and what it costs.

a) *Is this not CWE-807 plus a race plus known streaming bugs?*: This is the objection we consider most likely, which is why we state it ourselves. It is partly correct. B0 is a known generic race and is presented as a baseline, not a contribution. The mechanisms behind M1 and M2 are individually known: trusting client-supplied values is CWE-807 [11], and disconnect-related accounting gaps appear in a deployed gateway issue [14] and in the mitigation shipped by a metered-token proxy [15]. We claim no novelty at the primitive level.

What the paper adds is fourfold. A unified accounting model in which otherwise-unrelated failures are the same integrity property violated at different points of one lifecycle. A taxonomy separating synchronization, lifecycle commitment and usage authority as experimentally distinguishable dimensions. An M1 ablation, confirmed by exhaustive model checking, showing that solvency and accounting integrity are orthogonal and need different primitives — which does not follow from “there is a streaming bug”. And an M2 ablation isolating computation from authority, where an architecture that recounts correctly still leaks. We do not assert that practitioners believed a recount was sufficient; we show experimentally that in the evaluated architectures it was not.

XIII. THREATS TO VALIDITY

a) Internal: A single uunicorn worker in the base configuration makes gateway recount numbers an upper bound. The host is shared and Docker scheduling is not controlled. Local loopback adds a roughly 45 ms floor per request on this Windows and Docker Desktop configuration, which makes the gateway ratios conservative for small inputs. Leakage that depends on delivered tokens is sensitive to disconnect timing at high concurrency: the within-cell standard deviation of delivered tokens is exactly zero at concurrency 20 or below and about 0.011 at 50 and 100.

b) Construct: The B0 debit path is a frozen module predating the pluggable accounting backends and does not reference them, so backend parity covers M1 and M2 only and B0’s storage independence is untested. Detectability was found by our own audit to restate experimenter-assigned labels rather than measure evidence, so we withdrew it entirely rather than report a weaker version.

c) Formal: The specification is finite — two to three concurrent requests, two value chunks, unit pricing — and only safety properties are checked. No liveness property and no parameterized or inductive result is claimed. There is no refinement proof from specification to implementation. The model assumes strongly consistent accounting state, so the asynchronous family of Section X-E lies outside it and its results are labelled empirical only. Multi-category pricing and tokenizer disagreement are likewise unmodelled.

d) External: One serving stack, one small model on CPU, one prompt family. This establishes that the architectural conclusions are not artifacts of our own generator; it does not establish behaviour on commercial providers, and the cache-split nondeterminism is a property of this stack’s prefix cache rather than a claim about hosted APIs. The generator-dependent cells we report bound how far per-manipulation magnitudes travel. Reconciliation delays are injected rather than observed in production, so the asynchronous result is conditional and stated as such: if settlement lags authorization by more than the inter-arrival time, atomicity on the settlement path does not prevent over-serving. Pricing tiers are synthetic.

XIV. RESPONSIBLE DISCLOSURE AND ETHICS

This work produced no new vulnerability findings against any third-party, live or production system. All experiments

ran against the local testbed shipped with the artifact, whose harness defaults to `localhost` and contains no third-party endpoints. No credentials, real user data, or production billing APIs were used. The public artifacts cited as prior evidence — a GitHub issue [14] and a merged pull request [15] — were public before this work and are not our findings. Every attack in the artifact is paired with a defense, and the testbed is released so that operators can measure their own gateways.

XV. CONCLUSION

Client-side under-payment in LLM metering is best modelled as one accounting boundary with three placements: synchronization, commitment timing, and usage authority. Localizing each to a state transition predicts, and our measurements confirm, that integrity and solvency are closed by different primitives and that a server-side recount must be authoritative rather than merely computed. Exhaustive model checking establishes the M1 orthogonality over all four combinations rather than the two we implemented, and the same checking refuted our first specification before it confirmed the corrected one.

The result we did not expect concerns the baseline. B0 began as a concurrency race, and the asynchronous experiment showed that concurrency was incidental: what matters is whether authorization is atomically coupled to economic commitment on the path that authorizes service. Delayed settlement breaks that coupling with no concurrency whatsoever. The corrected condition is more general than the one we started with, and it is the form an operator should test against. In the tested configuration the strongest defense costs a small and predictable fraction of real inference time.

REFERENCES

- [1] G. Sun, Z. Wang, B. Tian, M. Liu *et al.*, “CoIn: Counting the invisible reasoning tokens in commercial opaque LLM APIs,” arXiv:2505.13778, May 2025.
- [2] G. Sun, Z. Wang, X. Zhao, B. Tian *et al.*, “Invisible tokens, visible bills: The urgent need to audit hidden operations in opaque LLM services,” arXiv:2505.18471, May 2025.
- [3] S. Hoque, J. Zhang, J. Sun, and F. Suya, “Token inflation: How dishonest providers can overcharge for large language model usage,” arXiv:2605.30040, May 2026.
- [4] D. Kelly, F. G. Glavin, and E. Barrett, “Denial of wallet — Defining a looming threat to serverless computing,” arXiv:2104.08031, Apr. 2021.
- [5] M. Dorsett, S. Mann, J. Chowdhury, and A. Mahmood, “A comprehensive review of denial of wallet attacks in serverless architectures,” arXiv:2508.19284, Aug. 2025.
- [6] OWASP GenAI Security Project, “LLM10:2025 unbounded consumption,” *OWASP Top 10 for LLM Applications*, 2025. [Online]. Available: <https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/>
- [7] Sysdig Threat Research Team, “LLMjacking: Stolen cloud credentials used in new AI attack,” Sysdig, May 2024. [Online]. Available: <https://www.sysdig.com/blog/llmjacking-stolen-cloud-credentials-used-in-new-ai-attack>
- [8] I. Shumailov, Y. Zhao, D. Bates, N. Papernot, R. Mullins, and R. Anderson, “Sponge examples: Energy-latency attacks on neural networks,” arXiv:2006.03463, Jun. 2020.
- [9] Y. Guan, “AEX: Non-intrusive multi-hop attestation and provenance for LLM APIs,” arXiv:2603.14283, Mar. 2026.
- [10] F. Wang and Z. Tian, “Evidence-bound gateway-path provenance for third-party LLM inference,” arXiv:2606.22560, Jun. 2026.
- [11] MITRE, “CWE-807: Reliance on untrusted inputs in a security decision,” Common Weakness Enumeration, version 4.20. [Online]. Available: <https://cwe.mitre.org/data/definitions/807.html>

- [12] J. Kettle, “Smashing the state machine: The true potential of web race conditions,” PortSwigger Research, 2023. [Online]. Available: <https://portswigger.net/research/smashing-the-state-machine>
- [13] P. E. O’Neil, “The escrow transactional method,” *ACM Trans. Database Syst.*, vol. 11, no. 4, pp. 405–430, Dec. 1986.
- [14] QuantumNous, “Streaming responses requests may refund all pre-consumed quota when final usage is missing after client disconnect,” new-api issue #5235, Jun. 2026. [Online]. Available: <https://github.com/QuantumNous/new-api/issues/5235>
- [15] Lightning Labs, “pricesrpc+proxy: Metered draw-down of prepaid L402 tokens, with a reference pricer daemon,” aperture pull request #247, merged Jul. 2026. [Online]. Available: <https://github.com/lightninglabs/aperture/pull/247>
- [16] WHATWG, “Server-sent events,” *HTML Living Standard*. [Online]. Available: <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [17] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” RFC 9110, Jun. 2022.
- [18] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” arXiv:1508.07909, Aug. 2015.
- [19] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” arXiv:1808.06226, Aug. 2018.
- [20] OpenAI, “tiktoken: A fast BPE tokeniser for use with OpenAI’s models,” software. [Online]. Available: <https://github.com/openai/tiktoken>
- [21] W. Kwon, Z. Li, S. Zhuang, Y. Sheng *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th Symp. Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [22] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” software. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [23] L. Ben Allal, A. Lozhkov, E. Bakouch, G. Martín Blázquez *et al.*, “SmolLM2: When smol goes big — Data-centric training of a small language model,” arXiv:2502.02737, Feb. 2025.
- [24] L. Lamport, “The temporal logic of actions,” *ACM Trans. Program. Lang. Syst.*, vol. 16, no. 3, pp. 872–923, May 1994.
- [25] Y. Yu, P. Manolios, and L. Lamport, “Model checking TLA+ specifications,” in *Correct Hardware Design and Verification Methods (CHARME)*, LNCS 1703, 1999, pp. 54–66.
- [26] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, “How Amazon Web Services uses formal methods,” *Commun. ACM*, vol. 58, no. 4, pp. 66–73, Apr. 2015.